

Authentication & Authorization Rearchitecture

FaceID Raspberry Pi Car Lock — CMPE 195 — Implementation Plan

Context

The Pi Device API currently authenticates every caller with **one shared master token** (`FACEID_API_TOKEN`). That closed the original hole — an unauthenticated `POST /api/users` from another host injected a user — but five problems remain: every client shares one secret, it is pasted by hand into each device, a leak grants total control, every token holder is a full admin, and a regular driver can read every other user's data.

Escalating finding from this inspection. The FastAPI Face API on port 8765 has **no authentication at all** and writes `users.face_encoding` on the *same* SQLite file. An attacker who reaches it can list enrolled names via `GET /api/face-status`, overwrite a real user's face template with their own via `POST /api/enroll/finish`, then stand in front of the camera and receive a genuine `UNLOCK`. The Flask token gate does not protect any of it.

Outcome: replace shared-token auth with per-user identities, one-time pairing, server-side sessions in `HttpOnly` cookies, and deny-by-default RBAC — and make the Pi the only network-facing service.

Architecture as traced

```
React PWA (Vite, plain HTTP origin)
  |
  |-- baseUrl      -> Flask Device API :5000 (0.0.0.0)  -- token gate --+
  |                  pi_device_api.py                               |
  |                                                           +--> db_api.py -> db.py ->
faceid.db
  +-- faceApiUrl -> FastAPI Face API :8765  NO AUTH AT ALL -----+
                    car_face_auth/src/api_server.py
```

Confirmed clean: no SQL injection — every value uses `?` placeholders. No XSS sinks — zero `innerHTML` / `dangerouslySetInnerHTML` / `eval`. No shell or subprocess use. ESP32 commands come from a fixed literal allowlist, never user input. The service worker deliberately never caches API responses.

Decisions taken

1. **Unify identity** — extend the existing `users` table; one person = one row = a face and a login.
2. **Fold the Face API in** — the Device API becomes the only network service; `:8765` binds loopback and takes an internal service token.
3. **Bootstrap via `install.sh`** — seeds an ADMIN row, prints a one-time pairing code.
4. **Five incremental milestones**, tests green at each.

Key architectural move: same-origin

The PWA is served from a *different* origin than the API today (`localhost:4173` vs `192.168.4.1:5000`). Cookies with `SameSite=Strict` are **not sent cross-site**, so cookie auth cannot work as-is. Rather than weaken cookies to `SameSite=None` (which forces HTTPS, CORS with credentials, and full CSRF token plumbing), make everything same-origin:

- **Production:** Flask serves the built `dist/` from the same origin as `/api` .
- **Development:** `vite.config.js` gains `server.proxy` forwarding `/api` → `http://127.0.0.1:5000` .

This *removes* code: CORS becomes a dev-only fallback, mixed-content warnings disappear, and CSRF drops to `SameSite=Strict` plus an `Origin` check rather than a token system. It is the simplifying change, not an added one.

Schema changes (db.py — additive only)

```
-- users: one new column. "enrollment status" derives from face_encoding IS NOT NULL.
ALTER TABLE users ADD COLUMN role TEXT NOT NULL DEFAULT 'USER'; -- 'ADMIN' | 'USER'

CREATE TABLE sessions (
  id          TEXT PRIMARY KEY, -- sha256 hex of raw token; plaintext never stored
  user_id     TEXT NOT NULL REFERENCES users(id),
  created_at  INTEGER NOT NULL,
  expires_at  INTEGER NOT NULL, -- absolute cap
  last_used_at INTEGER NOT NULL, -- idle timeout
  revoked     INTEGER NOT NULL DEFAULT 0,
  user_agent  TEXT              -- truncated, for the "your devices" list
);

CREATE TABLE pairing_codes (
  id          TEXT PRIMARY KEY, -- sha256 hex of the raw code
  user_id     TEXT NOT NULL REFERENCES users(id),
  created_by  TEXT REFERENCES users(id),
  created_at  INTEGER NOT NULL,
  expires_at  INTEGER NOT NULL, -- +5 min
  used_at     INTEGER           -- NULL until redeemed; single-use marker
);

CREATE INDEX idx_sessions_user    ON sessions(user_id);
CREATE INDEX idx_sessions_expires ON sessions(expires_at);
CREATE INDEX idx_pairing_expires  ON pairing_codes(expires_at);
CREATE INDEX idx_auth_logs_ts     ON auth_logs(ts); -- existing table, missing index
```

Hashing choice: sha256, not bcrypt/argon2. These are 256-bit values from `secrets.token_urlsafe(32)`, not human passwords — there is no dictionary to slow down, and a deliberately slow hash on every request is a real cost on a Pi.

Authentication flow

```
Pairing code (one-time, 5 min)
| POST /api/pair/redeem {code}
v
Backend: sha256(code) -> row; check unused, unexpired, rate limit
| mark used_at (single-use), create session row
v
Set-Cookie: faceid_session=<raw>; HttpOnly; SameSite=Strict; Path=/; [Secure]
|
v
Every later request: cookie -> sha256 -> row -> not revoked/expired/idle
-> load user -> active? -> g.user = {id, name, role}
```

- Session token `secrets.token_urlsafe(32)`; only its hash is stored.
- Idle timeout 12 h (`last_used_at`), absolute 30 d (`expires_at`).
- A fresh session row is created on redeem — no pre-auth identifier is ever reused.

- Logout deletes the row *and* expires the cookie.
- `Secure` flag set when `FACEID_REQUIRE_HTTPS=1`.

Authorization matrix

Centralized as `ENDPOINT_POLICY` in `auth.py`, evaluated in one `before_request`. **Unlisted view** → **ADMIN** (deny by default), plus a test asserting every registered route appears in the map, so a new route cannot ship without a deliberate decision.

Endpoint	Unauth	USER	ADMIN
<code>/health</code> , <code>/ready</code> (reduced payload)	yes	yes	yes
POST <code>/api/pair/redeem</code> (rate-limited)	yes	—	—
GET <code>/api/me</code> , POST <code>/api/auth/logout</code>	no	self	yes
GET <code>/api/status</code> , <code>unlock/lock/ignition-stop/full-reset</code>	no	yes	yes
POST <code>/api/scan/*</code>	no	yes	yes
GET <code>/api/users/<id></code>	no	self only	yes
POST <code>/api/enroll/*</code> , DELETE <code>/api/users/<id>/face</code>	no	self only	yes
GET/POST <code>/api/users</code> , DELETE <code>/api/users/<id></code>	no	403	yes
PATCH <code>.../access</code> , role changes	no	403	yes
GET <code>/api/logs</code> , GET/POST <code>/api/settings</code>	no	403	yes
POST <code>/api/pair/create</code>	no	403	yes
GET/DELETE <code>/api/sessions</code>	no	own only	all

IDOR defense: `require_self_or_admin(resource_user_id)` compares against `g.user.id` derived from the session — never from a request field. For a USER, enrollment ignores any client-supplied name and uses the session identity, so `POST /api/enroll/start {"name":"someone_else"}` cannot retarget.

Embeddings: `/api/face-status` returns the full enrolled list to ADMIN and only `{face_enrolled: bool}` to a USER. Raw vectors never leave the Pi; a test asserts no response body ever contains `face_encoding`.

Files to modify

File	Change
db.py	New tables, <code>role</code> column, indexes; <code>PRAGMA journal_mode=WAL</code> , <code>busy_timeout</code>
db_api.py	Session/pairing CRUD; fix <code>get_user_by_id</code> <code>SELECT *</code> → projection dropping <code>face_encoding</code>
auth.py NEW	All security logic: session issue/validate/revoke, pairing, <code>require_auth</code> , <code>require_role</code> , <code>require_self_or_admin</code> , rate limiter, <code>ENDPOINT_POLICY</code>
validation.py NEW	<code>require_str/bool/int</code> with length, range, allowlist
pi_device_api.py	Session gate replaces token gate; <code>/api/me</code> , <code>/api/auth/logout</code> , <code>/api/pair/*</code> , <code>/api/sessions</code> ; policy map; <code>MAX_CONTENT_LENGTH</code> ; 500 handler; security headers; serve <code>dist/</code>
car_face_auth/src/api_server.py	Loopback bind in code; internal service token; fix <code>allow_origins=["*"]</code> + <code>allow_credentials=True</code>
car_face_auth/src/pi_runtime.py	Enrollment bound to <code>user_id</code> , not a client-supplied name
src/hooks/useApi.js	<code>credentials:"include"</code> ; drop bearer token; 403 branch in <code>describeError</code>
src/hooks/useAppState.js	Purge <code>apiToken</code> from <code>localStorage</code> ; add <code>me / role</code>
src/hooks/useAppActions.js	<code>refresh()</code> → <code>Promise.allSettled</code> ; stop flipping <code>online:false</code> on 403
src/components/PairingScreen.jsx NEW	Redeem a code, establish session
ConnectionPanel.jsx, App.jsx, SidebarNav.jsx	Token field → pairing field; role-gate admin tabs (UI convenience, never the boundary)
vite.config.js, install.sh, systemd/	Dev proxy; seed ADMIN + print pairing code; service hardening
tests/test_pi_device_api.py	Full auth / authz / pairing / session / IDOR matrix

Milestones

1. **Backend identity core** — schema, `auth.py`, sessions, `/api/me`, policy map, RBAC, tests. Master token still accepted so nothing breaks yet.
2. **Pairing** — create/redeem, hashing, 5-min expiry, single-use, rate limiting; installer bootstrap.
3. **Frontend migration** — cookies, pairing screen, logout, role gating, `refresh()` fix, purge stored token.
4. **Face API lockdown** — loopback bind, internal token, CORS fix, reroute the 8 direct calls, serve `dist/`, vite dev proxy.
5. **Hardening** — `MAX_CONTENT_LENGTH`, 500 handler, validation, security headers + CSP, rate limits, log pruning, HTTPS config.

Master token disposition

Traced: it is used **only** for browser auth. The ESP32 talks over *serial*, not HTTP, and never sees a token. After milestone 3 it is repurposed as the internal service token for Flask → loopback Face API and is never accepted from the network for user routes.

Assumptions

- One Pi, a handful of phones; no multi-server scaling — so server-side sessions beat JWT (instant revocation, no signing key, no dependence on the Pi's batteryless clock).
- `SameSite=Strict` is acceptable because the app becomes same-origin.
- CSRF handled by `SameSite=Strict` + `Origin` checks rather than a token system — defensible for a same-origin LAN device, documented as a tradeoff.
- Existing `users` rows migrate to `role='USER'`; the installer seeds the first ADMIN.
- No new runtime dependency — `stdlib` `secrets` / `hashlib` only.

Known limitations to report at the end

- **No TLS shipped.** Cookies get `Secure` only when `FACEID_REQUIRE_HTTPS=1` behind a proxy the user configures. Over plain HTTP on an untrusted LAN, session cookies are interceptable. A deployment step, not something code can assert.
- **Pickle deserialization of face templates remains** (`face_engine.py:103`). Write access to `faceid.db` yields code execution. Milestone 4 removes the unauthenticated write path; migrating to `numpy .npy` is follow-up work.
- **Rate limiting is per-process, in-memory** — resets on restart.
- **`liveness` / `failLockout` are still not enforced by any code.** The UI advertises controls that do not exist; either implement or remove them.
- Flask's development WSGI server is still the runtime; a production WSGI server is separate work.

Completing this plan does not make the application “secure.” It closes the specific findings listed and establishes a structure where future routes are restricted by default. The limitations above remain open.